wednesday
21/10/26

Date AND Monday.

Different than rod cutting since this have constraint

LECTURE-16

19/10/26



Knap Sack: We've a sack and we need to maximise the profit by putting specific weight in sack.

- we'll have a sack with fixed weight capacity and have items which will have profit and weight. So we need to put those objects in the sack which will maximise profit.

Question:

i	1	2	3	4
w_i weight	2	5	7	3
price	20	30	32	10

$W = 9$ (max capacity of weight of sack)

$n = 4$

$$\sum_{i=1}^{k} w_i \leq W \quad \& \quad \max \sum_{i=1}^{k} P_i$$

Constraints

$\max$ ↓ maximise profit.

weight Constraint. Current item cannot be $\leq$ max weight

Base Case: current capacity = w (column heading of table) (n, 0)

Either capacity of sack is zero then profit is zero KS

OR all the items are in the sack (no more to add) KS(0, w)

so profit still zero.

General Case: if w_i (current item) $> w$

K (i, w)

cannot include w_i in sack as weight getting ≤ 0

So KnapSack (i-1, w)

else

price = max { can Include: KnapSack (i-1, w - w_i) + P[i] ; should not Includes: KnapSack (i-1, w) }

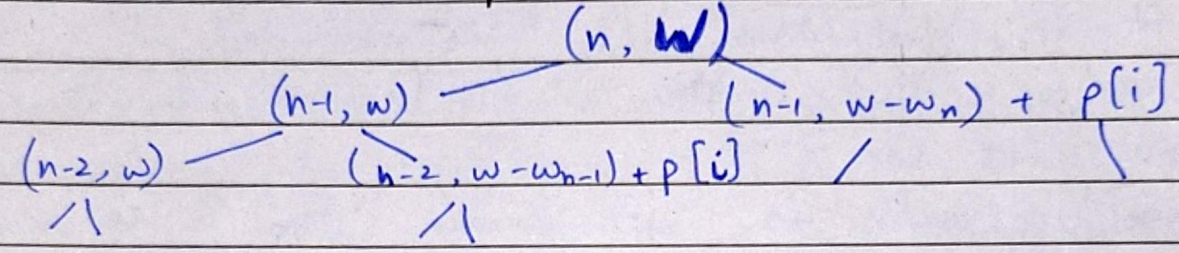
↓ as profit is not getting maximised.

$T(n) = x\,T(n-1) + O(n^{1/2/3})$

always gives x^n time complex.

Date

⇒ Divide & Conquer:



Height = n , leaf nodes = 2^n , $T(n) = \boxed{2^n}$ (exp) !!

⇒ Brute force:

Make all combinations and then check which solution satisfies the weight constraint. Then out of those; check which solution maximises the profit. This all cost $T(n) = \boxed{2^n}$:(.

⇒⇒ Dynamic Programming Solution: 2D array of $[(n+1) \times (W+1)]$

i ↓ W →	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	a 20	b 20	c 20	20	20	20	20	20	20
2	0	d 20	e 20	20	20	f 30	30	g 50	50	50
3	0	20	20	20	20	30	30	50	50	52
4	0	20	20	20	20	30	30	50	50	52

How to see which items to include (out of 1,2,3,4)?

- Start from maximum value/price in table i.e: 52 of bottom right corner.
- We'll see that whether that 52 in (4,9) came from inclusion of item 4 or exclusion of item 4.
- For checking that we'll see the above cell of that targetted cell.... so for (4,9) we'll see (3,9).

Date

- Since value of (4, 9) & (3, 9) is same i.e: 52. Therefore it tells that item 4 is NOT included.
- Now we'll see wheather item 3 will be included or not.
- For (3,9) we'll see (2, 9).
- The price is 52 and 50 which means item 3 is included as the price has changed from 50 → 52.
- We'll include item 3 and subtract the weight of item 3 i.e: $w_3 = 7$ from W (max weight) i.e 9

  9 - 7 = 2 (remaining weight capacity in sack).

- Now we'll switch to column 2 as leftover w is 2 (and above row) So we'll jump from (2, 9) → (1, 2)
- Now we'll see if item 1 needs to be included or not
- for that we'll check the above cell of (1,2) i.e: (0, 2)
- Price has changed from 20→0 as we moved from (1,2)→(0,2) means that item 1 is included.
- Do 2 - 2 = 0 (Base case: STOP)

  remaining weight of item 1.

⇒ Time Complexity = $O(n \cdot W)$

Since we are calculating table value with $n \times W$ cells and need to fill each cell ∴ time = $n \cdot W$.

→ Space Complexity: = $O(n \cdot W)$

since 2D array is of $(n+1 \times W+1)$ size.

CamScanner

we've two options; IF statement (cannot include) and ELSE statement (can include). In else statement; we can either include or exclude (should not include as profit not good).

Date

Method To fill 2D Array:

$i = 1$, $w_i = 2$, price $= 20$ (from table in question)

a: since $w = 1$ and $w_i = 2$ ∴ $w_i > w$ so we WONT include
∴ profit $= 0$. (from cell: (0, 0) as $(i-1, w)$ in IF block

b. $w = 2$ and $w_i = 2$ ∴ $w_i \not> w$ so move to ELSE block.

$(i, w) = (1, 2)$

exclude $(i-1, w)$ → $(0, 2)$ → profit$_{(0,2)}$ = 0

include $(i-1, w-w_i) + P[i]$ → $(0, 0) + P[1]$ → $0 + 20$ → profit$_{(0,0)}$ = 20

Max (0, 20) = 20 ✓

c: $w = 3$ & $w_i = 2$ ($3 \not> 2$ so else block)

$(i, w) = (1, 3)$

exclude $(i-1, w)$ → $(0, 3)$ = 0 at 0,3

include $(i-1, w-w_i) + P[i]$ → $(0, 1) + P[i]$ → $0 + 20$ = 20 at 0,1

max (0, 20) = 20 ✓

→ same for first row since 'exclude' always gives zero & max = 20

$i = 2$, $w_i = 5$, price $= 30$

d: since $w = 1$ and $w_i = 5$ ∴ $w_i > w$ so IF block & we wont include
∴ profit = 0 from cell: (1, 0) as $(i-1, w)$ in IF block

e: since $w = 2$ and $w_i = 5$ ∴ $w_i > w$ so IF block & we won't include ∴ profit = cell (1, 2) from $(i-1, w)$ = 20 ✓

* This will continue till $w = 5$ (column heading) as till then $w_i > w$.

Pearl Notes

CamScanner

Date

f: w = 5 , wi = 5 (wi ≠ w) so ELSE block.

(i=2, w=5)

exclude (i-1, w) → (1, 5) → profit = 20

include (i-1, w-wi) + P[i] → (1, 0) + 30 → profit = 0 + 30 = 30

max (20, 30) = 30 ✓

g: w = 7 , wi = 5 (wi ≠ w so else block)

(i=2, w=7)

exclude (i-1, w) → (1, 7) = 20

include (i-1, w-wi) + P[i] → (1, 2) + 30 → 20 + 30 = 50

max (20, 50) = 50 ✓

⇒ Top Down v/s Bottom Up (comes when we started DP).

	Memoization	Tabulation
• How do start?	• Starts with Fib(5), then Fib(4)...	• Calculate Fib(0) first & then Fib(1)...
• How do answers are calculated?	• Jab zaurat parayi dekhi jaye gi	• In a planned order all are calculated like solution 1, 2, 3.....
• How to save ans.?	• Array/dictionary	• Array/table
• Main Advantage	• May skips subproblems you don't need	• Avoids the recursion that is about to come ahead
• Type of Code	• Recursive	• Usually iterative/loops.

★ Memoization main start with problem right away & keep finding solutions while breaking down the problem (& keep saving in DP array sath sath)

Tabulation main sabsay choti problem say shuru ho kr build up krtay (while saving ans. in DP array) towards the main problem